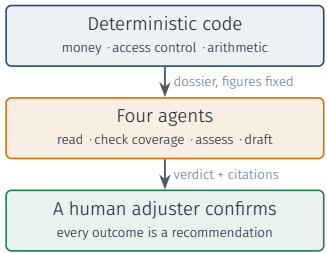
ClaimSense AI

A four-agent claims adjudicator
with a deterministic spine

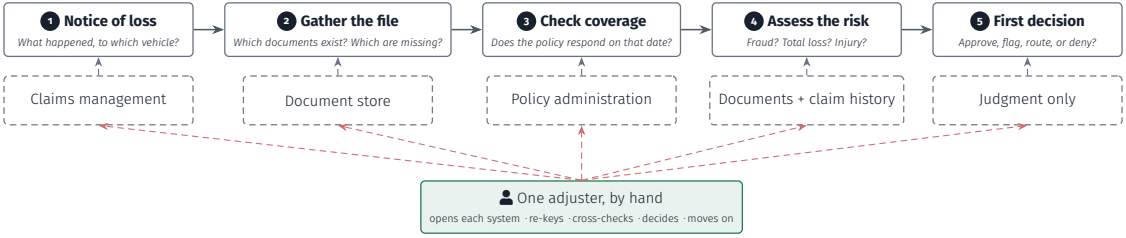
The problem · the approach · what you build

FDE capstone · weeks 13–17 · AWS + OpenAI Agents SDK



deterministic code model judgment human confirmation outside the project

What the project is about: a claim today, step by step



A Every step is manual

The three client systems do not talk to each other. The adjuster is the integration layer, and the queue waits on one pair of hands.

B Judgment is invisible

Two adjusters reach different decisions on the same file. The reasons live in an inbox, not in a record anyone can audit.

C Risk is a threshold

Fraud referral fires on a score. A 2:14 AM unwitnessed loss with a contradictory medical note looks like any other claim.

What the project is about: why a rule table is not enough

The file · CLM-2024-2822

incident date	31 Jan 2024
damage	pothole; tires, rim, control arm
repair estimate	$3,100
policy status field	**Active**
policy term	28 Jan 2023 – **28 Jan 2024**

Every value above was computed by code from the policy and claim records. Only the reading of them is judgment.

a few fields

A rule table reads fields

`status == Active` → approve
`fraud_score >= 0.5` → refer
`repair >= 0.75 * value` → total loss

⇒ **approve.** Paid on a lapsed policy.

everything

A reader reads the whole file

The loss is three days *after* the term ended. The status field is stale. The policy-period clause says the policy does not respond.

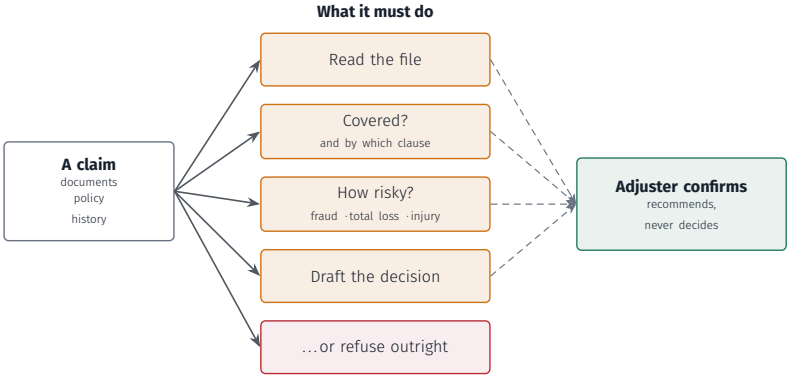
⇒ **deny,** citing `CLS-PER-006`. The adjuster confirms.

So the system has to

1. **Read the whole file**, not a handful of fields: narrative, documents, dates, history, and the contradictions between them.
2. **Ground every coverage statement** in the policy text, and cite the clause it came from.
3. **Explain the decision** in language an adjuster and an auditor can check.
4. **Never touch money or access.** Payout arithmetic and who-may-see-what stay in code, where they are exact and testable.
5. **Leave the decision to a human.** The system drafts a recommendation; the adjuster confirms it.

That is the job description for a language model wrapped in a deterministic spine.

The brief: what the client is asking for



Given, not chosen

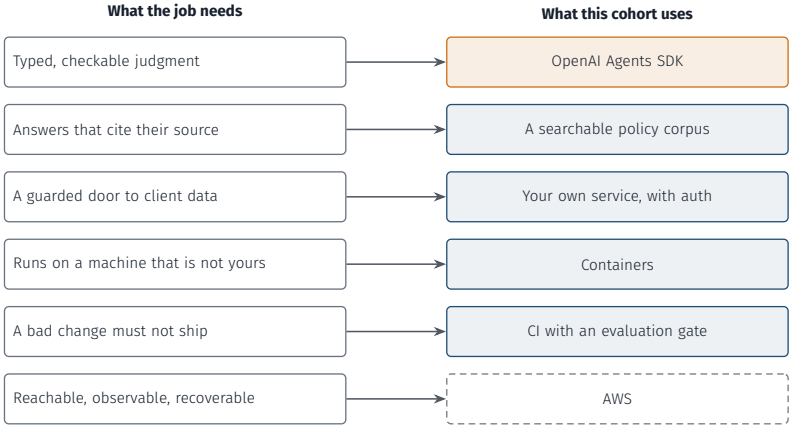
Code, never a model
identity · access · dates · money

Model, always cited
every judgment shows what it rested on

Synthetic data only
nothing real touches any cloud

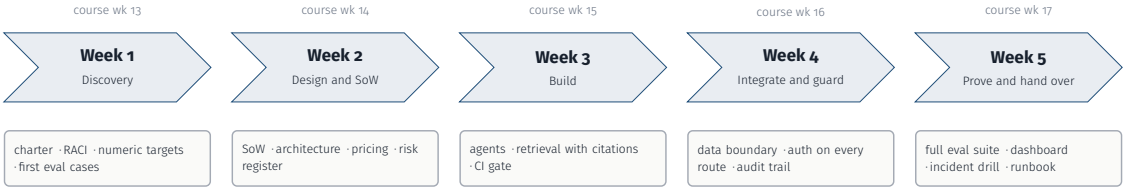
Minutes, not hours, with the reasoning written down and a person still accountable. Everything else is yours to design in weeks 1 and 2.

The standard stack: what the job needs, what this cohort uses



Python runtime, web framework of your choosing, synthetic fixtures supplied. You will need an AWS account, an OpenAI key, Docker, Python 3.12+ and Node 20+. The same job has been built on GCP with Gemini and on Azure with Claude.

What you are expected to do: one milestone per capstone week



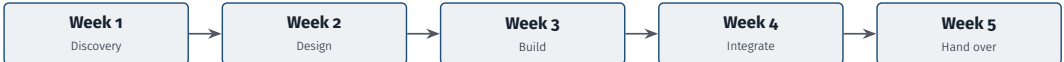
These are engagement milestones

Not class assignments. The Sunday session walks the stage and unblocks you; the milestone is what you ship into your repo that week. Weekly feedback is formative; the score comes once, at the end.

Two rules for all five weeks

A verdict your rule table could have produced does not count as a model decision, and you must be able to tell them apart. Access control is code that runs before any agent, never a prompt.

Deadlines: two submissions across the five weeks



Form 1 · Checkpoint

end of week 2, before the week 3 class

Your weeks 1 and 2 artefacts, and a screenshot of the stack running under your own account. Rough is fine.

Not scored. It checks you are moving and not stuck.

Form 2 · Final

end of week 5

Three assets: the codebase at a `final` tag; the observation document; the link to your deployed frontend, with a demo recording.

Scored out of 100. See the next slide.

In between

Weekly artefacts live in your repo and get written feedback. They are not scored on their own; the final submission is expected to contain them.

Evidence for the final submission

Deploy your solution on AWS, then capture it. High-quality screenshots of everything you stood up: the running console, CloudWatch dashboards and alarms, the container registry, log and audit lines, the CI run. Annotate every screenshot so a reader knows what they are looking at, and submit the set with your final.